

TJSC 2026 - Analista de Sistemas

Infraestrutura de TI e Computação em Nuvem

Aula 8

Virtualização, hypervisor, máquinas virtuais, containers e diferença entre VM e container

Material original, orientado ao edital vigente do TJSC 2026 e calibrado para cobrança da banca FGV.

Padrão: teoria aprofundada, exemplos práticos de dev/API/backend/cloud, tabelas comparativas, pegadinhas, 20 questões autorais, gabarito separado e comentários completos.

Campo	Definição nesta aula
Disciplina	Infraestrutura de TI e Computação em Nuvem - 8 questões no edital
Aula da trilha	Aula 8 de 13
Foco	Virtualização, hypervisor, máquinas virtuais, containers e comparação VM x container
Perfil considerado	Desenvolvedor Java/Quarkus, Angular, TypeScript, APIs e sistemas corporativos, com necessidade de aprender infraestrutura sem pular base
Limite da aula	Docker, registries, volumes e redes serão aprofundados na Aula 9; Kubernetes e Rancher ficam na Aula 10

Como estudar: esta aula fecha a ponte entre servidor físico e cloud. Você deve sair sabendo explicar o que o hypervisor faz, por que uma VM não é um container, por que container não é simplesmente 'uma VM leve' e quais trade-offs a FGV tende a cobrar.

1. Identificação da aula e recorte do edital

Esta aula cobre a parte da trilha que liga infraestrutura física/lógica, sistemas operacionais, virtualização e containers. No edital, ela se conecta principalmente aos itens de infraestrutura de TI, servidores, armazenamento e virtualização, além de desenvolvimento para nuvem e containers. O recorte é deliberado: explicar virtualização e containers como fundamento; Docker e orquestração serão tratados nas aulas seguintes.

Tópico da trilha	Cobertura nesta aula	Limite para não invadir aula futura
Virtualização	Conceito, objetivos, consolidação de servidores, isolamento, abstração de recursos, virtualização de CPU, memória, disco e rede	Não aprofundar cloud service models, que ficam na Aula 11
Hypervisor	Tipo 1 e tipo 2, papel do Virtual Machine Monitor, relação com host, guest, hardware e sistema operacional	Não transformar em aula de produto específico
Máquinas virtuais	Guest OS, vCPU, memória virtual da VM, disco virtual, NIC virtual, snapshots, templates, clones e live migration em visão de prova	Não aprofundar backup/storage, já tratado na Aula 7
Containers	Isolamento em nível de sistema operacional, namespaces, control groups, imagem, runtime, kernel compartilhado e portabilidade	Docker detalhado fica na Aula 9; Kubernetes e Rancher na Aula 10
VM x container	Diferenças de arquitetura, isolamento, peso, boot, portabilidade, segurança, densidade, uso típico e pegadinhas	Sem marketing cloud-native

2. Como a FGV costuma cobrar este assunto

Em provas correlatas de tecnologia, a FGV costuma explorar virtualização e containers por meio de comparação entre conceitos próximos. O enunciado raramente exige detalhe de implementação profundo; a dificuldade vem de alternativas que trocam o nível de abstração: VM virtualiza hardware; container isola processos usando o kernel do host; hypervisor não é sistema operacional convidado; snapshot não é backup completo; orquestrador não é runtime de container.

Padrão de cobrança	Como aparece	Risco para o candidato
Associação de conceito	Identificar hypervisor, VM, container, imagem, runtime, host e guest	Chamar container de VM leve ou confundir imagem com container em execução
Comparação técnica	Escolher entre VM e container para cenário de isolamento, portabilidade ou densidade	Responder pelo termo da moda, não pelo requisito técnico
Afirmativas I, II e III	Misturar vantagens verdadeiras com exageros: 'container sempre é mais seguro', 'VM sempre é mais lenta'	Cair em absolutismos
V/F	Separar hypervisor tipo 1, tipo 2, guest OS, kernel compartilhado e isolamento	Confundir sistema operacional host com guest
Caso prático	Ambiente corporativo quer consolidar servidores, testar versões, empacotar API ou rodar OS diferente	Não perceber se a necessidade é virtualização completa ou empacotamento de aplicação

3. Siglas, termos e leitura técnica

Sigla/termo	Expansão	Inferência de prova
VM	Virtual Machine	Máquina virtual com hardware virtual e, normalmente, sistema operacional convidado próprio
VMM	Virtual Machine Monitor	Outro nome para hypervisor; camada que cria e controla VMs
vCPU	Virtual Central Processing Unit	CPU virtual atribuída à VM; não é necessariamente um núcleo físico dedicado

Sigla/termo	Expansão	Inferência de prova
Guest OS	Guest Operating System	Sistema operacional convidado executado dentro da VM
Host	Máquina hospedeira	Servidor físico ou sistema que hospeda VMs/containers
NIC	Network Interface Card	Interface de rede; pode existir como NIC física ou NIC virtual
NAT	Network Address Translation	Pode aparecer em redes virtuais de hypervisors, mas não é sinônimo de virtualização
COW	Copy-on-Write	Técnica comum em snapshots/clones; escreve apenas diferenças após um ponto inicial
OCI	Open Container Initiative	Padroniza formatos e runtimes de containers
cgroups	control groups	Mecanismo Linux para limitar e contabilizar recursos como CPU e memória
namespace	espaço de nomes isolado	Mecanismo Linux que isola visão de processos, rede, mounts, usuários etc.
Runtime	ambiente de execução	Componente que efetivamente cria/executa o container a partir da imagem

4. Virtualização: ideia central

Virtualizar é criar uma abstração de um recurso computacional real. Em vez de uma aplicação conversar diretamente com todo o hardware físico, uma camada intermediária apresenta recursos virtuais: CPU virtual, memória virtual, disco virtual, interface de rede virtual. Para prova, a virtualização permite executar múltiplos ambientes isolados sobre a mesma infraestrutura física, com melhor aproveitamento de recursos e maior flexibilidade operacional.

Na sua realidade de dev, pense assim: antes de uma API Quarkus chegar ao Kubernetes, ela pode rodar em uma VM Linux em um data center corporativo. Essa VM enxerga um disco, uma interface de rede e CPUs, mas esses recursos são abstrações controladas pelo hypervisor. A aplicação não precisa saber se está em servidor físico ou virtual, desde que o ambiente entregue CPU, memória, rede, disco e sistema operacional compatíveis.

Objetivo da virtualização	Descrição	Pegadinha
Consolidação	Rodar múltiplos servidores lógicos no mesmo hardware físico	Não significa rodar sem sistema operacional
Isolamento	Separar ambientes, falhas e configurações entre VMs	Isolamento não é absoluto nem dispensa segurança
Portabilidade	Mover ou replicar VMs com menor dependência do hardware físico	Portabilidade de VM é diferente de portabilidade de container
Elasticidade operacional	Criar, desligar, clonar ou migrar ambientes com mais agilidade	Elasticidade cloud exige automação e capacidade disponível
Teste e homologação	Criar ambientes padronizados para validação	Snapshot ajuda retorno rápido, mas não substitui backup

4.1 Virtualização não é só 'rodar uma máquina dentro de outra'

A frase ajuda, mas é incompleta. Em prova, virtualização envolve abstração e gerenciamento de recursos. O hypervisor intercepta ou coordena operações sensíveis, aloca memória, agenda execução de vCPUs, apresenta dispositivos virtuais e conecta interfaces virtuais a redes virtuais. A VM tem a ilusão de possuir hardware próprio, ainda que esse hardware seja implementado por software e mapeado para recursos físicos compartilhados.

Distinção importante: a virtualização de servidores cria máquinas virtuais completas; a virtualização em nível de sistema operacional, típica de containers, cria ambientes isolados de processos que compartilham o mesmo kernel do host. Essa diferença é o coração da aula.

5. Hypervisor: o controlador das máquinas virtuais

Hypervisor, também chamado de Virtual Machine Monitor (VMM), é a camada que cria, executa e gerencia máquinas virtuais. Ele fica entre as VMs e os recursos físicos ou entre as VMs e um sistema operacional hospedeiro, conforme o tipo. Sua função não é executar a regra de negócio da aplicação, mas fornecer e controlar o ambiente virtual onde os sistemas operacionais convidados rodam.

5.1 Hypervisor tipo 1 e tipo 2

Critério	Tipo 1 - bare metal	Tipo 2 - hosted
Onde executa	Diretamente sobre o hardware físico	Sobre um sistema operacional hospedeiro
Exemplos típicos	VMware ESXi, Microsoft Hyper-V em papel de servidor, KVM em distribuições Linux corporativas	VirtualBox, VMware Workstation, Parallels Desktop
Uso comum	Data center, servidores corporativos, cloud, alta disponibilidade	Laboratório, desenvolvimento, testes locais
Desempenho/control e	Tende a oferecer maior controle, desempenho e integração com recursos corporativos	Mais simples para uso local, mas depende do SO hospedeiro
Pegadinha FGV	Tipo 1 não precisa de um sistema operacional hospedeiro convencional entre ele e o hardware	Tipo 2 não significa que a VM não tenha guest OS

A FGV pode cobrar a frase 'hypervisor executado diretamente sobre o hardware'. Essa é pista forte de tipo 1. Se o enunciado fala em uma aplicação de virtualização instalada no Windows, Linux ou macOS do usuário para laboratório, a pista tende a ser tipo 2.

5.2 Host, guest e hardware virtual

O host é a máquina hospedeira ou a plataforma que fornece recursos. O guest é o sistema operacional convidado que roda dentro da VM. A VM enxerga hardware virtual: vCPU, memória, disco virtual e interface de rede virtual. O hypervisor faz a mediação entre esse hardware virtual e os recursos físicos.

Termo	O que é	Exemplo
Host	Ambiente que hospeda as VMs	Servidor físico com hypervisor
Guest OS	Sistema operacional dentro da VM	Ubuntu Server rodando API Quarkus
vCPU	CPU virtual apresentada à VM	2 vCPUs alocadas para servidor de aplicação
vNIC	Interface de rede virtual	Adaptador da VM ligado a switch virtual
VMDK/VHD/VHDX/QCOW2	Formatos de disco virtual, conforme tecnologia	Arquivo ou volume que representa disco da VM

5.3 Virtualização de CPU, memória, disco e rede

A virtualização não se limita à CPU. O candidato precisa entender que a VM consome um conjunto de recursos virtuais que o hypervisor mapeia para recursos físicos. Isso explica por que **overcommit pode aumentar densidade, mas também gerar contenção.**

Recurso	Como aparece para a VM	Ponto de prova
CPU	vCPUs	vCPU não é necessariamente núcleo físico exclusivo; o hypervisor agenda execução
Memória	RAM atribuída à VM	Ballooning, swap e overcommit podem afetar desempenho; não confundir com memória virtual do SO guest
Disco	Disco virtual ou volume	Pode usar arquivo, LUN, datastore; snapshot de disco não é backup completo

Recurso	Como aparece para a VM	Ponto de prova
Rede	vNIC ligada a switch/bridge virtual	A VM pode usar rede NAT, bridge, VLAN, firewall virtual e roteamento

6. Máquina virtual: o que ela é e o que ela não é

Máquina virtual é um ambiente computacional isolado que simula uma máquina completa. Em regra, cada VM possui seu próprio sistema operacional convidado, bibliotecas, serviços e aplicações. Para a aplicação Java ou para o banco, parece haver um servidor comum; por baixo, os recursos são virtuais.

Uma VM não é apenas um processo comum sem sistema operacional próprio. A VM roda um guest OS. Esse é um ponto que cai muito em comparação com containers: **containers empacotam aplicações e dependências em processos isolados que compartilham o kernel do host; VMs executam sistemas operacionais convidados inteiros sobre hardware virtual.**

6.1 Ciclo de vida de VMs

Recurso	Função	Limite/pegadinha
Template	Modelo padronizado para criar VMs semelhantes	Não é backup operacional por si só
Clone	Cópia de uma VM existente	Pode exigir ajustes de identidade, rede, SID/hostname
Snapshot/checkpoint	Estado em ponto no tempo para retorno rápido	Não substitui backup; pode degradar desempenho se mantido indevidamente
Live migration	Movimentar VM em execução entre hosts compatíveis	Depende de storage/rede/cluster; não é simplesmente copiar arquivo
High availability	Reiniciar ou mover VM em caso de falha de host	HA da VM não garante disponibilidade interna da aplicação se o app estiver mal projetado

6.2 Snapshot não é backup

Snapshot é uma fotografia lógica do estado de uma VM ou de seu disco em determinado momento. Ele é útil para rollback antes de uma atualização, patch ou alteração de configuração. Mas ele pode depender do mesmo datastore, do mesmo storage e da mesma cadeia de discos diferenciais. Se o storage for perdido, ou se a cadeia for corrompida, o snapshot pode não salvar a operação. Backup exige cópia, retenção, restauração testada e, dependendo do requisito, imutabilidade/offsite.

Essa distinção conversa diretamente com a Aula 7: RAID não substitui backup; snapshot também não substitui backup. A FGV adora esse tipo de troca de finalidade.

7. Containers: isolamento em nível de sistema operacional

Container é uma unidade de execução isolada que empacota aplicação e dependências necessárias para rodar de modo consistente em diferentes ambientes. A grande diferença em relação à VM é que o container não traz um sistema operacional convidado completo com kernel próprio. Em ambientes Linux, ele usa o kernel do host e obtém isolamento por mecanismos como namespaces e control groups.

Para sua realidade de dev: uma API Quarkus pode ser empacotada em uma imagem de container. Ao executar essa imagem, surge um container, que é o processo isolado em execução. A imagem é o pacote; o container é a instância em execução. A VM é mais ampla: nela você instala um sistema operacional inteiro e depois executa aplicações dentro dele.

7.1 Namespaces e cgroups

Mecanismo	Função	Exemplo de efeito
Namespaces	Isolam a visão que o processo tem de recursos do sistema	Processos em um container enxergam seu próprio espaço de processos, rede, mounts, hostname etc.
cgroups - control groups	Limitam, contabilizam e controlam uso de recursos	Limitar CPU, memória, I/O e outros recursos de um container
Capabilities	Reduzem privilégios do processo mesmo quando parece root dentro do container	Evitar que root no container tenha todos os poderes do root do host
seccomp/AppArmor/SELinux	Filtros e políticas adicionais de segurança	Restringir chamadas de sistema ou perfis de acesso

Em prova, a associação mais provável é: namespaces isolam visão de recursos; cgroups limitam e contabilizam recursos. Se uma alternativa diz que cgroups 'fornecem nomes DNS' ou que namespaces 'limitam consumo de CPU', leia com cuidado: há troca de função.

7.2 Imagem, container e runtime

A imagem de container é um artefato imutável ou quase imutável, composto por camadas, com aplicação, dependências e metadados. O container é a instância em execução dessa imagem. O runtime é o componente que cria e executa o container, aplicando isolamento, filesystem, rede e processo inicial. O Docker popularizou essa experiência, mas o padrão aberto mais cobrável em nível conceitual é a Open Container Initiative (OCI), que busca padronizar imagens e runtimes.

Conceito	Definição curta	Erro clássico
Imagem	Pacote da aplicação e dependências	Achar que imagem é sempre processo em execução
Container	Instância em execução de uma imagem	Achar que container inclui kernel próprio completo
Runtime	Componente que executa containers	Confundir com orquestrador
Registry	Repositório de imagens	Confundir com banco de dados da aplicação

8. VM x container: comparação que precisa estar automática

Critério	Máquina virtual - VM	Container
Unidade virtualizada	Hardware virtual completo	Ambiente de processo isolado no SO
Sistema operacional	Normalmente cada VM tem seu guest OS	Compartilha o kernel do host
Kernel	Pode ter kernel diferente do host, conforme virtualização	Usa o kernel do host; não traz kernel próprio completo
Isolamento	Mais forte por fronteira de hardware virtual/guest OS	Mais leve, mas depende do isolamento do kernel e configuração
Inicialização	Mais lenta, pois sobe SO convidado	Mais rápida, pois inicia processo isolado
Tamanho	Geralmente maior	Geralmente menor
Densidade	Menor densidade no mesmo host, em regra	Maior densidade, em regra
Uso típico	Rodar sistemas operacionais diferentes, legado, isolamento forte, consolidação de servidores	Empacotar e escalar aplicações, microserviços, CI/CD, cloud-native
Pegadinha	VM não é container pesado por definição; é virtualização de máquina	Container não é VM leve no sentido técnico; é isolamento em nível de SO

8.1 Quando escolher VM

- Quando é necessário executar sistemas operacionais distintos ou kernels diferentes no mesmo hardware.
- Quando a aplicação legada depende de ambiente completo e difícil de containerizar.
- Quando o isolamento forte entre workloads é requisito central.
- Quando a organização já possui plataforma madura de virtualização com backup, HA e governança de VMs.
- Quando o objetivo é consolidar servidores tradicionais, e não necessariamente empacotar microserviços.

8.2 Quando escolher container

- Quando o foco é empacotar aplicação e dependências de forma portátil e reproduzível.
- Quando a aplicação segue arquitetura mais modular, stateless ou cloud-native.
- Quando se busca inicialização rápida, alta densidade e integração com pipelines de CI/CD.
- Quando o ambiente alvo usa orquestração de containers, como Kubernetes.
- Quando não é necessário carregar um sistema operacional convidado completo por instância.

9. Isolamento e segurança: o ponto em que a FGV complica

Um erro comum é transformar comparação técnica em propaganda. Não é correto afirmar que containers são sempre inseguros nem que VMs são sempre seguras. VMs tendem a oferecer isolamento mais forte por virtualizarem hardware e executarem guest OS separado. Containers oferecem isolamento mais leve, dependente do kernel compartilhado e das políticas de segurança configuradas. Em contrapartida, containers são mais leves, iniciam rápido e favorecem padronização de deploy.

Afirmção	Avaliação de prova
Container compartilha kernel com o host	Correta em arquitetura típica Linux; é uma diferença central para VM
Container é sempre uma VM mais leve	Imprecisa; container não virtualiza uma máquina completa
VM sempre impede qualquer escape	Errada; hypervisors também podem ter vulnerabilidades
Root dentro do container pode ser perigoso	Correta; requer redução de privilégios, user namespaces, capabilities e políticas
Imagem oficial ou popular é automaticamente segura	Errada; imagem deve ser atualizada, verificada e escaneada

10. Fluxo aplicado: da sua API ao ambiente virtualizado

Cenário: uma equipe do setor público precisa publicar uma API Java/Quarkus para consulta de processos. Existem três caminhos técnicos plausíveis, cada um com leitura de prova diferente.

Caminho	Como funciona	Leitura de prova
Servidor físico	Instala Linux diretamente no hardware e executa a API	Menos abstração; menor flexibilidade de provisionamento
VM	Cria uma VM Linux sobre hypervisor, instala Java, dependências e aplicação	Virtualização de hardware; guest OS próprio; bom para consolidação e isolamento
Container	Empacota a API e dependências em imagem e executa como processo isolado	Isolamento em nível de SO; kernel compartilhado; mais leve e portátil

A pergunta de prova pode vir assim: 'o ambiente precisa executar Windows Server e Linux no mesmo host físico'. Isso aponta para VMs. Ou: 'a equipe deseja empacotar a aplicação com suas

dependências e iniciar múltiplas instâncias rapidamente'. Isso aponta para containers. Ou ainda: 'a solução deve orquestrar réplicas, pods e services'. Aí o tema já é Kubernetes, que será Aula 10.

11. Pegadinhas FGV

Pegadinha	Como resolver
Container é uma VM leve	Como simplificação informal até aparece, mas tecnicamente é perigoso. Container isola processos e compartilha kernel; VM virtualiza hardware e roda guest OS.
Hypervisor é o sistema operacional convidado	Errado. Hypervisor gerencia VMs; guest OS roda dentro da VM.
Hypervisor tipo 2 roda direto no hardware	Errado. Essa é a pista de tipo 1. Tipo 2 roda sobre SO hospedeiro.
Toda VM exige um host físico dedicado	Errado. A virtualização permite múltiplas VMs no mesmo host físico.
Snapshot substitui backup	Errado. Snapshot serve para ponto de retorno rápido; backup exige cópia recuperável e estratégia de retenção/restore.
Container possui kernel próprio completo	Errado em arquitetura típica: compartilha kernel do host.
Imagem e container são sinônimos	Errado. Imagem é artefato; container é instância em execução.
Orquestrador é runtime	Errado. Runtime executa container; orquestrador coordena implantação, escala e estado desejado.
VMs são obsoletas por causa de containers	Errado. VMs e containers atendem requisitos diferentes e coexistem em cloud e data centers.

12. Resumo conceitual sem decorar demais

Virtualização completa cria máquinas virtuais com hardware virtual e, em regra, sistema operacional convidado próprio. Hypervisor é a camada que controla essa execução. Containers empacotam aplicação e dependências e executam processos isolados usando recursos do kernel do host. VMs favorecem isolamento forte e diversidade de sistemas operacionais; containers favorecem leveza, velocidade, densidade e padronização de deploy. Em prova, responda pelo requisito do cenário, não pelo termo mais moderno.

13. Lista de questões autorais - estilo FGV

Instrução: resolva as 20 questões sem consultar o gabarito. O bloco mistura questão direta, caso prático, afirmativas, V/F, relacione, exceção e interpretação de cenário. As alternativas foram construídas com distratores próximos, sem cópia literal de questões reais.

Questão 1

Em um ambiente de data center, deseja-se executar múltiplos servidores lógicos, cada um com sistema operacional próprio, sobre o mesmo hardware físico. A camada responsável por criar e controlar essas máquinas virtuais é denominada:

- (A) registry.
- (B) namespace.
- (C) hypervisor.
- (D) control group.
- (E) service mesh.

Questão 2

Um hypervisor instalado como aplicação sobre um sistema operacional hospedeiro, utilizado principalmente em laboratório ou estação de trabalho de desenvolvimento, é classificado como:

- (A) hypervisor tipo 1, pois qualquer hypervisor executa diretamente sobre o hardware.
- (B) container runtime, pois executa aplicações isoladas sem sistema operacional convidado.
- (C) bare metal, pois elimina a necessidade de sistema operacional host.
- (D) kernel guest, pois substitui o sistema operacional convidado.
- (E) hypervisor tipo 2, pois depende de um sistema operacional hospedeiro.

Questão 3

Assinale a alternativa que melhor expressa a diferença essencial entre uma máquina virtual e um container.

- (A) A máquina virtual normalmente possui sistema operacional convidado próprio, enquanto o container compartilha o kernel do host em uma arquitetura típica.
- (B) O container sempre virtualiza hardware completo, enquanto a máquina virtual apenas empacota bibliotecas de aplicação.
- (C) A máquina virtual não utiliza CPU, memória ou disco, pois todos os recursos são fornecidos pela aplicação.
- (D) O container exige, necessariamente, um sistema operacional convidado diferente do host.
- (E) VM e container são tecnicamente equivalentes, variando apenas o fornecedor da solução.

Questão 4

Uma equipe deseja executar, no mesmo servidor físico, uma aplicação legada que depende de Windows Server e outra aplicação que depende de Linux com kernel específico. A solução mais aderente, em tese, é:

- (A) usar apenas cgroups para limitar CPU das duas aplicações no mesmo kernel.
- (B) converter os dois sistemas em uma única imagem de container com dois kernels.
- (C) usar apenas um proxy reverso, pois ele virtualiza sistemas operacionais.
- (D) criar máquinas virtuais separadas, cada uma com seu sistema operacional convidado.
- (E) usar RAID 10, pois ele isola sistemas operacionais por espelhamento.

Questão 5

Sobre snapshots de máquinas virtuais, assinale a afirmativa correta.

- (A) São sempre equivalentes a backups completos externos e imutáveis.
- (B) Podem registrar um ponto de retorno do estado da VM, mas não devem ser tratados, por si sós, como substitutos de uma estratégia de backup.
- (C) Eliminam a necessidade de armazenamento persistente.
- (D) Funcionam apenas em containers e não se aplicam a VMs.
- (E) Impedem qualquer degradação de desempenho independentemente do tempo de retenção.

Questão 6

Analise as afirmativas a seguir. I. Em uma VM, o sistema operacional convidado enxerga recursos virtuais, como vCPU, memória e disco virtual. II. Em uma arquitetura típica de containers Linux, cada container executa um kernel próprio completo. III. O hypervisor coordena a execução de VMs sobre recursos físicos ou sobre um sistema hospedeiro, conforme o tipo. Está correto o que se afirma em:

- (A) I, apenas.
- (B) II, apenas.
- (C) I e III, apenas.
- (D) II e III, apenas.
- (E) I, II e III.

Questão 7

Em containers Linux, os mecanismos que, respectivamente, isolam a visão de recursos do sistema e controlam/limitam o consumo de recursos são:

- (A) hypervisors e LUNs.
- (B) VLANs e NAT.
- (C) snapshots e templates.
- (D) forks e sockets.
- (E) namespaces e cgroups.

Questão 8

Considere as afirmações sobre imagem e container. Assinale V para a verdadeira e F para a falsa. () Imagem é um artefato usado como base para criar containers. () Container é uma instância em execução, derivada de uma imagem. () Registry é o processo isolado em execução no kernel do host. A sequência correta é:

- (A) F - V - V.
- (B) V - V - F.
- (C) V - F - V.
- (D) F - F - V.
- (E) V - V - V.

Questão 9

Relacione os conceitos às descrições. 1. Hypervisor 2. Guest OS 3. Container 4. Imagem de container () Sistema operacional executado dentro da máquina virtual. () Artefato empacotado usado para instanciar containers. () Processo isolado que compartilha o kernel do host em arquitetura típica. () Camada que cria e gerencia máquinas virtuais. A sequência correta é:

- (A) 2 - 4 - 3 - 1.
- (B) 1 - 3 - 4 - 2.

- (C) 4 - 2 - 1 - 3.
- (D) 3 - 1 - 2 - 4.
- (E) 2 - 3 - 4 - 1.

Questão 10

Um órgão utiliza VMs para consolidar servidores. Em determinado host, foram atribuídas mais vCPUs às VMs do que núcleos físicos disponíveis, assumindo que nem todas usarão pico simultaneamente. Essa prática está relacionada a:

- (A) paridade distribuída.
- (B) criptografia TLS.
- (C) armazenamento em objeto.
- (D) overcommit de recursos.
- (E) resolução de nomes por DNS.

Questão 11

Assinale a alternativa INCORRETA sobre máquinas virtuais e containers.

- (A) Uma VM pode executar sistema operacional convidado próprio.
- (B) Containers tendem a iniciar mais rapidamente que VMs, pois não inicializam um guest OS completo por instância.
- (C) Containers sempre oferecem isolamento mais forte que VMs, em qualquer cenário e configuração.
- (D) VMs podem ser usadas para consolidar servidores em data centers.
- (E) Containers favorecem empacotamento e portabilidade de aplicações.

Questão 12

Uma equipe cria um modelo padronizado de VM com sistema operacional, pacotes básicos e configuração inicial, para acelerar o provisionamento de novos servidores. Esse modelo é mais adequadamente chamado de:

- (A) namespace.
- (B) template.
- (C) cgroup.
- (D) registry.
- (E) pod.

Questão 13

No contexto de virtualização, live migration é melhor descrita como:

- (A) conversão de imagem de container em arquivo de backup imutável.
- (B) substituição automática de RAID 5 por RAID 6 durante falha de disco.
- (C) consulta DNS feita antes de iniciar uma VM.
- (D) execução de um container sem rede.
- (E) movimentação de uma VM em execução entre hosts compatíveis, com impacto operacional reduzido, quando a plataforma suporta tal recurso.

Questão 14

Um analista afirma que 'container é mais leve que VM porque cada container possui seu próprio sistema operacional completo, porém sem usar CPU'. A avaliação correta é:

- (A) A afirmação está errada: containers são leves justamente por não carregarem um guest OS completo por instância e por compartilharem o kernel do host em arquitetura típica.
- (B) A afirmação está correta, pois containers não usam CPU.
- (C) A afirmação está correta apenas para containers Windows, pois todo container ignora o kernel.
- (D) A afirmação está correta, pois VM não possui sistema operacional.
- (E) A afirmação está errada apenas porque containers não podem executar aplicações web.

Questão 15

Considere um ambiente em que uma API stateless precisa ser empacotada com suas dependências e executada em múltiplas instâncias com inicialização rápida, mantendo consistência entre desenvolvimento, homologação e produção. A tecnologia mais alinhada a esse objetivo é:

- (A) HBA Fibre Channel.
- (B) RAID 1.
- (C) Servidor torre com DAS.
- (D) Containers.
- (E) Hypervisor tipo 2 exclusivamente local.

Questão 16

Sobre hypervisors tipo 1, é correto afirmar que:

- (A) são sempre instalados como extensões de navegador.
- (B) não podem executar máquinas virtuais com sistemas operacionais diferentes.
- (C) executam diretamente sobre o hardware físico, sendo comuns em ambientes corporativos e de data center.
- (D) são sinônimos de containers e dispensam hardware virtual.
- (E) funcionam apenas para armazenar imagens em registry.

Questão 17

Em relação à segurança de containers, assinale a alternativa mais adequada.

- (A) O compartilhamento do kernel do host torna dispensáveis políticas de segurança adicionais.
- (B) Todo container executado como root é automaticamente seguro por estar isolado.
- (C) Imagens populares dispensam atualização e verificação de vulnerabilidades.
- (D) A segurança depende de configuração, redução de privilégios, controle de recursos, isolamento e gestão adequada das imagens.
- (E) Containers não usam processos do sistema operacional.

Questão 18

Um candidato afirma que o orquestrador de containers é o componente que executa diretamente a syscall de criação do container no kernel, substituindo o runtime. A afirmação está:

- (A) correta, porque orquestrador e runtime são sempre o mesmo componente.
- (B) correta, porque Kubernetes é apenas um formato de imagem.
- (C) errada, porque o runtime executa containers, enquanto o orquestrador coordena implantação, escala e estado desejado.
- (D) correta apenas em VMs, pois VMs não usam hypervisor.
- (E) errada apenas porque containers não podem ser orquestrados.

Questão 19

Na virtualização de servidores, uma vNIC é:

- (A) uma chave criptográfica usada em TLS.
- (B) uma interface de rede virtual apresentada à VM e conectada a estruturas de rede virtual ou física por intermédio do hypervisor/plataforma.
- (C) um disco local diretamente conectado ao servidor.
- (D) um protocolo de backup incremental.
- (E) um tipo de memória persistente que substitui RAM.

Questão 20

Observe o cenário: um servidor físico executa um hypervisor tipo 1. Sobre ele há três VMs Linux. Dentro de uma das VMs, a equipe executa containers de uma API. A leitura técnica correta é:

- (A) os containers eliminam a existência do hypervisor físico.
- (B) as VMs compartilham obrigatoriamente o mesmo kernel do host físico.
- (C) as VMs são containers, pois ambos usam IP.
- (D) há camadas distintas: hypervisor virtualiza máquinas; a VM fornece um sistema operacional convidado; os containers isolam processos dentro do kernel disponível naquela VM.
- (E) a arquitetura é impossível, pois containers nunca podem rodar dentro de VMs.

14. Gabarito resumido

O gabarito foi produzido de forma orgânica, sem quota matemática de letras. Se errar, classifique o erro como: conceito, confusão entre termos, leitura apressada, pegadinha FGV ou lacuna de teoria.

Questões	Gabarito
1 a 5	1-C, 2-E, 3-A, 4-D, 5-B
6 a 10	6-C, 7-E, 8-B, 9-A, 10-D
11 a 15	11-C, 12-B, 13-E, 14-A, 15-D
16 a 20	16-C, 17-D, 18-C, 19-B, 20-D

15. Comentários completos

Questão 1 - Gabarito: C

Hypervisor, ou Virtual Machine Monitor, é a camada que cria e controla VMs. A alternativa A é tentadora se você lembrar de containers, mas registry armazena imagens. B e D são mecanismos associados a containers Linux, não à criação de VMs completas. E é conceito de malha de serviços, fora do papel de virtualização.

Questão 2 - Gabarito: E

Hypervisor tipo 2 roda sobre um sistema operacional hospedeiro. A alternativa A descreve tipo 1. B confunde virtualização de máquina com runtime de container. C usa 'bare metal', típico de tipo 1. D inventa 'kernel guest' como substituto do hypervisor.

Questão 3 - Gabarito: A

A distinção central é guest OS próprio na VM versus kernel compartilhado pelo container. B inverte as definições. C nega recursos óbvios de VM. D erra porque container típico não exige kernel convidado. E elimina diferenças técnicas relevantes.

Questão 4 - Gabarito: D

Se há necessidade de sistemas operacionais diferentes e kernels específicos, VMs são a solução natural. A falha porque cgroups limitam recursos dentro do mesmo kernel. B erra ao falar em container com dois kernels. C confunde proxy com virtualização. E mistura RAID com isolamento de sistemas operacionais.

Questão 5 - Gabarito: B

Snapshot ajuda a voltar a um estado anterior, mas não substitui backup. A exagera e transforma snapshot em backup externo imutável. C confunde snapshot com armazenamento persistente. D é falso: snapshots são comuns em VMs. E usa absoluto indevido; snapshots podem ter impacto operacional.

Questão 6 - Gabarito: C

I e III estão corretas. II está errada porque container típico não executa kernel próprio completo; ele compartilha o kernel do host. A alternativa correta exige perceber a troca entre VM e container.

Questão 7 - Gabarito: E

Namespaces isolam a visão de recursos; cgroups limitam e contabilizam recursos. A mistura hypervisor e storage. B traz rede, mas não isolamento de container. C fala em snapshots/templates. D usa conceitos de processo e comunicação, mas não é a dupla clássica de container.

Questão 8 - Gabarito: B

Imagem é artefato; container é instância em execução. Registry é repositório de imagens, não o processo isolado. A, C, D e E erram porque atribuem ao registry a função de container ou negam o papel da imagem/container.

Questão 9 - Gabarito: A

Guest OS é o sistema operacional da VM; imagem é artefato; container é processo isolado com kernel compartilhado; hypervisor gerencia VMs. As demais sequências trocam artefato, execução e camada de controle.

Questão 10 - Gabarito: D

Atribuir mais vCPU do que núcleos físicos esperando que nem todas usem pico simultâneo é overcommit. A é RAID/storage. B é segurança de transporte. C é modelo de storage. E é DNS.

Questão 11 - Gabarito: C

A incorreta é C: containers não oferecem sempre isolamento mais forte que VMs. Eles podem ser adequados e seguros, mas o isolamento depende do kernel compartilhado e da configuração. A, B, D e E estão corretas em visão de prova.

Questão 12 - Gabarito: B

Template é modelo padronizado para criar VMs. Namespace e cgroup são mecanismos de container. Registry armazena imagens. Pod é unidade de Kubernetes, tema aprofundado depois.

Questão 13 - Gabarito: E

Live migration é movimentar VM em execução entre hosts compatíveis com impacto reduzido, quando a plataforma suporta. As demais alternativas misturam container, RAID, DNS e execução sem rede.

Questão 14 - Gabarito: A

Container é mais leve porque não carrega guest OS completo por instância e compartilha kernel do host em arquitetura típica. B é absurdo técnico porque containers usam CPU. C erra ao dizer que ignora kernel. D nega SO em VM. E inventa impossibilidade de aplicação web.

Questão 15 - Gabarito: D

Empacotar API stateless com dependências, executar múltiplas instâncias e manter consistência de ambientes é caso típico de containers. HBA, RAID e DAS são infraestrutura/storage. Hypervisor tipo 2 local não é a melhor leitura para produção com múltiplas instâncias.

Questão 16 - Gabarito: C

Hypervisor tipo 1 executa diretamente sobre hardware e é comum em data center. A é absurda. B é falsa: pode executar VMs com sistemas operacionais diferentes. D confunde VM com container. E confunde hypervisor com registry.

Questão 17 - Gabarito: D

Segurança de containers exige configuração: privilégios mínimos, isolamento, controle de recursos, imagens atualizadas e políticas. A, B e C usam absolutos perigosos. E é falso: containers executam processos.

Questão 18 - Gabarito: C

Runtime cria/executa container; orquestrador coordena implantação, escala e estado desejado. A e B confundem papéis. D mistura com VMs. E nega orquestração, justamente o tema que será aprofundado com Kubernetes.

Questão 19 - Gabarito: B

vNIC é interface de rede virtual apresentada à VM. A é TLS, C é DAS/disco, D é backup, E é memória. A questão testa se o candidato entende que hardware virtual inclui rede.

Questão 20 - Gabarito: D

É possível rodar containers dentro de uma VM. O hypervisor virtualiza máquinas; a VM tem guest OS; containers isolam processos usando o kernel disponível dentro daquela VM. A, B, C e E confundem camadas ou negam uma arquitetura comum.

16. Checklist final de domínio

- Consigo explicar hypervisor tipo 1 e tipo 2 sem confundir com sistema operacional convidado.
- Consigo definir VM como máquina com hardware virtual e, em regra, guest OS próprio.
- Consigo explicar container como processo isolado que compartilha o kernel do host em arquitetura típica.
- Consigo diferenciar imagem, container, runtime e registry.
- Consigo dizer por que snapshot não é backup.
- Consigo escolher VM ou container a partir do requisito do cenário.
- Consigo identificar o erro em 'container é uma VM leve' quando a prova exigir rigor técnico.
- Consigo associar namespaces a isolamento e cgroups a controle/limite de recursos.

17. Fontes técnicas e observações

Este material é original e não reproduz questões reais. Provas FGV correlatas foram usadas apenas para calibrar estrutura, densidade e estilo de cobrança. As referências técnicas abaixo sustentam a terminologia e os conceitos centrais da aula.

Fonte	Uso na aula
Edital TJSC n. 10/2026, retificado em 10/04/2026	Base do conteúdo programático e da distribuição de questões
Manual Mestre de Calibração FGV para Aulas e Questões	Padrão editorial, profundidade, variedade de questões e comentários
NIST SP 800-125 - Guide to Security for Full Virtualization Technologies	Virtualização completa, hypervisor, máquinas virtuais e preocupações de segurança
Docker Docs - What is Docker? / What is a Container?	Conceito de containers, imagem e empacotamento de aplicação
Open Container Initiative	Padronização de formatos e runtimes de containers
Linux Kernel Documentation - Control Group v2	cgroups como mecanismo de controle de recursos
Kubernetes documentation	Referência contextual sobre workloads containerizados, sem antecipar a Aula 10
Microsoft Learn - Hyper-V	Exemplo técnico de hypervisor e virtualização corporativa
Provas FGV correlatas de TI	Calibração do estilo: Docker, containers, Kubernetes, afirmações V/F e conceitos próximos

Próxima aula da trilha: Aula 9 - Docker: imagens, containers, registries, volumes e redes.